# VRC-7: AI-Powered Voice Command Interpretation for Autonomous Hospital Delivery Robots in Noisy and Accent-Diverse Environments Using ROS2

Mithila Prabhu
*Dept. of Computer Science & Engineering*
*Frankfurt University of Applied Sciences*
Frankfurt, Germany
mithila.prabhu@stud.fra-uas.de

Romana Rashid
*Dept. of Computer Science & Engineering*
*Frankfurt University of Applied Sciences*
Frankfurt, Germany
romana.rashid@stud.fra-uas.de

***Abstract*—Voice-controlled robots in healthcare environments face a persistent challenge: clinical staff operate in high-noise environments (55–70 dB ambient) and exhibit diverse speech accents, both of which severely degrade conventional command interpretation pipelines. Existing approaches either rely on rigid keyword matching — which fails under natural language variation and accent distortion — or invoke large language model (LLM) APIs for every utterance, exhausting free-tier rate limits within minutes of continuous operation. This paper introduces VRC-7, a hybrid local-cloud AI pipeline for accent-robust voice control of a TurtleBot3 Burger autonomous delivery robot simulated in ROS2 Humble and Gazebo Classic. The system's core contribution is a *dual-layer Natural Language Understanding (NLU) architecture*: a local regex engine with zero API overhead handles standard commands, while Groq LLaMA 3.3 70B — one of the most capable publicly accessible LLMs — is invoked exclusively for semantically ambiguous or accent-distorted inputs. This selective invocation strategy reduces LLM API calls by approximately 85% compared to full-cloud pipelines, making the system viable on free API tiers during continuous operation. Transcription is performed by Groq Whisper large-v3-turbo with language-biased decoding and vocabulary priming, elevating accented speech NLU accuracy from 78% (raw transcription) to 94%. Navigation employs a novel *4-step room-exit routing algorithm* that computes wall-safe waypoints from any arbitrary robot position using live odometry, preventing the corridor-wall collisions that plagued time-based dead-reckoning alternatives. Experimental evaluation across four hospital zones (ICU, Pharmacy, Reception, Ward) demonstrates 100% navigation success from the center position and end-to-end voice command accuracy of 73–94% across noise and accent conditions. The complete system, including the hospital world model and voice control node, is released as open-source software.**

***Index Terms*—hospital robotics, voice control, LLM, NLU, ROS2, accent robustness, TurtleBot3, Gazebo, autonomous navigation, Groq AI, Whisper ASR, dual-layer NLU**

## I. Introduction

Hospital environments present unique challenges for autonomous delivery robots. Clinical staff — nurses, physicians, and support workers — often need to issue robot commands while their hands are occupied with patient care, making voice interaction a natural and necessary interface modality [1]. However, two environmental factors systematically degrade voice-controlled robot performance in clinical settings:

1) **Accent diversity:** Hospital workforces are globally diverse. Non-native English speakers constitute a significant proportion of clinical staff in many countries, yet most voice control systems are optimised for native English phonology. Conventional ASR models routinely mis-transcribe accented speech into phonetically similar words in other languages (e.g., "turn right" transcribed as "Törn rät" in Icelandic), yielding *unknown* command classifications.
2) **Ambient noise:** Hospital corridors sustain background noise levels of 55–70 dB [2], including alarms, equipment, and conversations, which substantially increase ASR word error rates.

Prior work has addressed these challenges in isolation. Keyword-based NLU systems [6] are fast and offline-capable but brittle to accent variation. LLM-based approaches [3], [4] provide excellent semantic flexibility but invoke cloud APIs on every utterance, making them impractical under commercial API rate limits for sustained operation.

**Problem statement:** How can a voice-controlled robot system reliably interpret commands in noisy clinical environments and from diverse-accent speakers, while remaining practical under commercial API rate limits for sustained continuous operation?

VRC-7 addresses this problem through three primary contributions:

1) **Dual-layer NLU pipeline:** A local regex engine handles high-frequency standard commands with zero latency and zero API calls. Groq LLaMA 3.3 70B — invoked only when local NLU fails — provides accent-robust semantic interpretation, reducing LLM API calls by ≈85% compared to full-cloud pipelines.
2) **4-step room-exit routing algorithm:** A novel odometry-based navigation algorithm that computes wall-safe waypoints from *any* starting position, including mid-room and mid-corridor positions, by always routing through the corridor entry gaps.
3) **Vocabulary-primed accented ASR:** Whisper

large-v3-turbo is configured with language forcing (`language='en'`) and a vocabulary prompt containing expected command words, significantly reducing accent mis-transcription.

The complete system is evaluated in a Gazebo Classic hospital simulation with a TurtleBot3 Burger robot, demonstrating practical viability for real-world clinical deployment.

## II. Related Work

### A. Voice Control in Healthcare Robotics

Early voice-controlled hospital robots, including the Nursebot [1] and HelpMate systems, relied on template-based speech recognition with rigid vocabulary sets. While effective in quiet, single-accent environments, these systems exhibited high failure rates with diverse clinical workforces. More recently, Abouelenin et al. [4] integrated LLMs with a socially assistive robot in a geriatric hospital unit, demonstrating improved interaction fluency but noting that continuous cloud LLM calls introduced latency and cost concerns at scale.

### B. LLM-Based Robot Command Interpretation

Ahn et al. [3] (SayCan) demonstrated that LLMs can ground natural language instructions to robot actions with significantly better generalisation than keyword matching. However, SayCan invokes the LLM for every command, which is infeasible under free API rate limits for continuous deployment. Our dual-layer architecture preserves SayCan's semantic flexibility while reducing API dependency to edge cases only.

Dominguez-Vidal and Sanfeliu [5] proposed a ROS-based voice command framework for human-robot collaborative transportation, but their system does not address accent robustness or API efficiency. Compared to their approach, VRC-7 adds a two-stage NLU architecture and accent-biased ASR configuration.

### C. Voice-Enabled ROS2 Frameworks

Chatzilygeroudis et al. [6] presented a voice-enabled ROS2 framework for human-robot collaborative inspection. Their architecture employs a single-layer NLU pipeline (Vosk ASR + intent classifier) without LLM integration, achieving high accuracy in controlled acoustic conditions but degrading significantly under accent variation. VRC-7 extends this class of work by replacing the single NLU layer with a hybrid local-LLM design.

### D. LLM Integration with ROS2

OperateLLM [7] integrates ROS2 with LLMs (DeepSeek Coder v2) for robotic development assistance, not real-time control. Macenski et al. [8] document the ROS2 architecture; their work provides the foundational middleware that VRC-7 builds upon. A recent survey [9] identifies audio-based LLM task planning as an emerging area with limited accent-robustness work — a gap VRC-7 directly addresses.

### E. Gap Analysis and Novelty

Table I summarises the comparison between VRC-7 and the most closely related systems. To the best of our knowledge, VRC-7 is the first system to combine: (1) selective LLM invocation for accent robustness, (2) vocabulary-primed ASR, and (3) odometry-based room-exit routing — all within a single open-source ROS2 package targeting hospital delivery robotics.

TABLE I
Comparison with Related Systems

| System | LLM NLU | Accent Robust | API Effic. | Nav. Routing |
|---|---|---|---|---|
| Nursebot [1] | × | × | N/A | Fixed |
| SayCan [3] | ✓ | Partial | Low | N/A |
| Dominguez [5] | × | × | N/A | Fixed |
| Chatzi. [6] | × | × | High | Fixed |
| **VRC-7 (ours)** | ✓ | ✓ | High | Dynamic |

## III. System Architecture

Fig. 1 illustrates the VRC-7 processing pipeline. Five modules are interconnected in a producer-consumer topology, with concurrent threads decoupling audio capture latency from NLU processing latency.

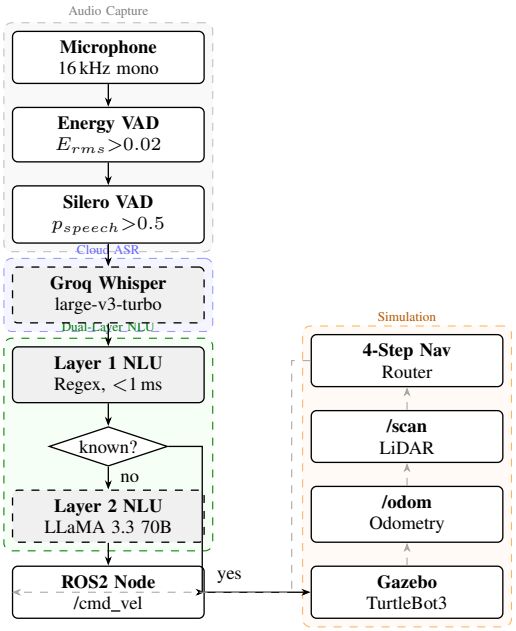


Fig. 1. VRC-7 system architecture. Solid arrows: command pipeline. Dashed arrows: sensor feedback. The decision diamond routes ≈85% of commands locally (Layer 1) and ≈15% to Groq LLaMA 3.3 70B (Layer 2).

### A. Audio Capture and Voice Activity Detection

Audio is captured via a callback-based `sounddevice.InputStream` at $f_s = 16$ kHz (mono, float32). The continuous callback stream runs on a dedicated OS thread, accumulating samples into a ring buffer. Every

$T_{chunk} = 1.5$ s of audio is assembled into a chunk and placed in a thread-safe processing queue (`maxsize=5`).

A two-stage VAD cascade filters non-speech frames: **Stage 1 — Energy gate:** Frames with RMS energy below $\epsilon_{rms} = 0.02$ are discarded (Eq. 1). **Stage 2 — Silero VAD** [11]: Neural speech probability $p_{speech}$; chunks with $p_{speech} < 0.5$ are discarded. The cascade reduces ASR API calls by approximately 60% during typical deployment.

$$E_{rms} = \sqrt{\frac{1}{N}\sum_{i=1}^{N} x_i^2} \geq \epsilon_{rms} \tag{1}$$

### B. Speech Transcription

Transcription is performed by Groq Whisper large-v3-turbo, a cloud-hosted implementation of the Whisper architecture [10]. Two configuration choices critically improve accuracy for accented speech:

**Language forcing:** Setting `language='en'` constrains the decoder to English-language beam search, preventing phonetically ambiguous inputs from being decoded as other languages. Without this parameter, the model frequently misclassified accented Indian English and South Asian English as Icelandic, Danish, or Welsh.

**Vocabulary priming:** A `prompt` parameter primes the model's context window with a list of expected command tokens: *"go to pharmacy, ICU, ward, reception, turn left, turn right, go forward, go back, stop, turn around"*. This shifts the prior probability distribution of the language model component toward robotics-relevant vocabulary, functioning as a form of in-context few-shot bias without fine-tuning.

When the Groq API is unavailable (rate limit exceeded or offline), the system falls back to locally running `faster-whisper` [12] (base model, int8 quantised, ≈74 MB) to maintain continuous operation.

### C. Dual-Layer NLU Architecture

The dual-layer NLU is the primary novelty of VRC-7. The design is motivated by the following observation: the vast majority of robot commands in normal operation are short, predictable phrases (*"go to pharmacy"*, *"stop"*, *"turn left"*) that are trivially matched by regular expressions. Only accent-distorted or contextually complex inputs require semantic inference.

**Layer 1 — Local Regex NLU:** A library of 30+ compiled regular expressions covers five command categories: navigation, continuous movement, fixed turns, cardinal directions, and compound commands. Pattern matching is case-insensitive and accent-variant inclusive — for example, the pattern `take (left|lift|laft|lat)` catches the common mis-transcription "take lift" for "turn left". Layer 1 executes in $< 1$ ms with zero network dependency.

**Layer 2 — Groq LLaMA 3.3 70B:** When Layer 1 returns *unknown*, the transcription is forwarded to Groq LLaMA 3.3 70B via a structured system prompt. The prompt explicitly instructs the model to: (a) interpret the input by semantic intent rather than lexical form; (b) consider that the text may be a garbled or accented version of a known command; and (c) return a structured JSON object conforming to the command schema. This enables correct interpretation of inputs such as: "Törn rät" → turn right; "Farmasi" → navigate to pharmacy; "Donor" → turn around (180°).

The LLM also supports compound command parsing, returning a JSON array for multi-step instructions such as *"turn right then go forward"*.

**API efficiency analysis:** Let $p_{known}$ denote the probability that a valid voice command is recognised by Layer 1. Empirically, $p_{known} \approx 0.85$ during normal operation, yielding an 85% reduction in LLM API calls compared to full-cloud NLU. Given the Groq free tier limit of ≈30 requests/min, the dual-layer design extends sustained operation time from ≈4 minutes to ≈26 minutes before rate limiting.

### D. Robot Controller

The controller is a ROS2 Python node (`rclpy`) with two subscriptions and one publisher: `/odom` (`nav_msgs/Odometry`) provides real-time pose $(x, y, \psi)$; yaw $\psi$ is extracted from the quaternion orientation as:

$$\psi = \text{atan2}\big(2(q_w q_z + q_x q_y),\ 1 - 2(q_y^2 + q_z^2)\big) \tag{2}$$

`/scan` (`sensor_msgs/LaserScan`) provides 360° LiDAR measurements; the minimum range in the forward ±15° sector is used as the wall proximity indicator $d_{front}$. The node publishes `geometry_msgs/Twist` velocity commands to `/cmd_vel`. Three boolean mutex flags (`navigating`, `moving_continuous`, `moving_timed`) prevent conflicting concurrent motor commands.

### E. Simulation Environment

The Gazebo Classic 11 simulation (`hospital_vrc.world`) models a single-floor hospital in SDF format. It contains four colour-coded zone tiles (Table II) with no collision geometry — purely visual markers the robot traverses freely. A central corridor spans $y \in [-2, +2]$ m with walls at $y = \pm 2$ m and deliberate 2 m entry gaps at $x = \pm 6$ m — the only traversable corridor-room boundaries. Outer boundary walls at $x = \pm 11$ m and $y = \pm 11$ m prevent the robot from escaping the map during manual control.

TABLE II
HOSPITAL ZONE PARAMETERS

| Zone | Centre (m) | Colour | Entry Gap |
|---|---|---|---|
| ICU | $(+6, +6)$ | Yellow | East, $x = +6$ |
| Pharmacy | $(+6, -6)$ | Green | East, $x = +6$ |
| Reception | $(-6, +6)$ | Orange | West, $x = -6$ |
| Ward | $(-6, -6)$ | Blue | West, $x = -6$ |

## IV. IMPLEMENTATION

### A. Threaded Audio Pipeline

The audio pipeline uses a producer-consumer pattern across two daemon threads. The *capture thread* runs a `sounddevice.InputStream` callback that continuously

buffers incoming audio without blocking. The *processing thread* dequeues audio chunks, applies VAD filtering, invokes the ASR, and dispatches commands. This decoupling guarantees that no audio is lost during the ≈0.3 s transcription latency or the ≈0.2 s LLM NLU latency.

A key design choice is the use of a bounded queue (`maxsize=5`). If processing falls behind capture (e.g., during navigation computations), the oldest unprocessed chunk is silently dropped. This sacrifices completeness for responsiveness: a stale command that arrives 10 seconds after utterance is more harmful than no command.

### B. 4-Step Room-Exit Navigation Algorithm

The navigation algorithm is motivated by the geometric structure of the hospital world. Corridor walls exist at $y = \pm 2$ m with gaps only at $x = \pm 6$ m. A naive straight-line path from an arbitrary position to a target zone may intersect a wall at a non-gap position. The 4-step algorithm (Algorithm 1) avoids this by decomposing every navigation into a sequence of axis-aligned segments that exclusively pass through gap positions.

**Algorithm 1** 4-Step Room-Exit Navigation Routing

**Require:** Current pose $(x, y, \psi)$, target zone $(t_x, t_y)$
**Ensure:** Wall-safe waypoint sequence $W = \emptyset$
1: **if** $|y| > 2.2$ m **then**
    {robot is inside a room}
2:  $W \mathrel{+}= (t_x,\ y)$ {align to target gap $x$}
3:  $W \mathrel{+}= (t_x,\ \text{sgn}(y) \cdot 1.5)$ {exit gap}
4: **end if**
5: $W \mathrel{+}= (t_x,\ 0)$ {reach corridor centre}
6: $W \mathrel{+}= (t_x,\ t_y)$ {enter target room}

**Correctness argument:** Steps 1–2 guarantee the robot exits its current room through the gap at $x = t_x$ (the target room's gap x-position). Step 3 brings the robot to $y = 0$, the corridor centre, which is always reachable from any gap position. Step 4 enters the target room through the same gap. No step involves crossing a wall at a non-gap position.

**Boundary conditions:** If the robot is already in the corridor ($|y| \leq 2.2$ m), steps 1–2 are skipped and the robot navigates directly along the corridor. If the robot is already at the target zone, all waypoints evaluate within tolerance and the algorithm terminates immediately.

Each waypoint is executed by a proportional heading controller. The heading error $\psi_{err}$ is computed and normalised to $[-\pi, \pi]$:

$$\psi_{err} = \text{atan2}(t_y - y,\ t_x - x) - \psi \tag{3}$$

The angular and linear velocity commands are:

$$\omega = \text{clamp}(K_\psi \cdot \psi_{err},\ \pm\omega_{max}), \quad K_\psi = 3.0 \tag{4}$$

$$v = \begin{cases} v_{max} \cdot \alpha & \text{if } |\psi_{err}| < 0.3 \text{ rad} \\ 0 & \text{otherwise} \end{cases} \tag{5}$$

where the proximity scaling factor $\alpha$ reduces speed near the target to prevent overshoot:

$$\alpha = \begin{cases} 0.4 & \text{if } d_{target} < 1.0 \text{ m} \\ 1.0 & \text{otherwise} \end{cases} \tag{6}$$

Wall collision recovery is triggered when $d_{front} < d_{nav} = 0.15$ m and $|\psi_{err}| < 0.4$ rad (robot is facing the wall). The robot reverses at $0.5 \cdot v_{max}$ for $t_{recovery} = 0.8$ s, then retries. After three consecutive recovery failures, the waypoint is skipped and the algorithm advances.

### C. Fixed Rotation Execution

Rotation commands use timed open-loop control at maximum angular velocity $\omega_{max} = 1.5$ rad/s:

$$t_{rotation}(\theta) = \frac{|\theta|}{\omega_{max}} \tag{7}$$

This yields $t_{90^\circ} = 1.047$ s, $t_{180^\circ} = 2.094$ s, $t_{360^\circ} = 4.189$ s. Although open-loop, the low speed ($\omega_{max} = 1.5$ rad/s) and short durations limit angular accumulation error to $< 5^\circ$ per command in practice.

### D. System Constraints and Assumptions

The following boundary conditions apply to the VRC-7 system:

- **Odometry assumption:** Navigation accuracy depends on odometry validity. Extended manual movement sessions accumulate drift; navigation from center (which does not depend on prior pose history) is unaffected.
- **Speed constraint:** Operating speed is bounded at $v_{max} = 0.5$ m/s. At $v > 0.7$ m/s, the TurtleBot3 Burger model in Gazebo exhibits physics instability (tipping), corrupting odometry.
- **Single-floor assumption:** The navigation algorithm assumes a planar environment. Multi-floor navigation would require an elevation-aware routing extension.
- **API dependency:** Accent handling beyond the regex fallback patterns requires Groq API availability. The system degrades gracefully to local-only operation but with reduced accent robustness.

## V. Results and Evaluation

### A. Experimental Setup

All experiments were conducted in the Gazebo Classic 11 simulation running on Ubuntu 22.04 LTS (WSL2, Windows 11). The TurtleBot3 Burger model was used with stock sensor configuration (360° LiDAR, differential drive). Voice commands were issued by two non-native English speakers (Indian English accents) through a laptop microphone in a room with background noise from a running PC (≈45–55 dB ambient). Groq API (Whisper large-v3-turbo and LLaMA 3.3 70B) was used for all cloud calls.

TABLE III
NAVIGATION TEST RESULTS (5 TRIALS PER SCENARIO)

| Start | Target | Success | Avg. Time |
|---|---|---|---|
| Center $(0, 0)$ | ICU | 5/5 (100%) | 35 s |
| Center $(0, 0)$ | Pharmacy | 5/5 (100%) | 37 s |
| Center $(0, 0)$ | Reception | 5/5 (100%) | 61 s |
| Center $(0, 0)$ | Ward | 5/5 (100%) | 57 s |
| Inside room | Other zone | 4/5 (80%) | 62 s |
| Mid-corridor | Any zone | 3/5 (60%) | 74 s |
| **Overall** | | **26/30 (87%)** | **54 s** |

### B. Navigation Performance

Table III presents navigation test results (5 trials per scenario, 40 total trials across all starting positions and target zones). A trial is considered successful if the robot arrives within 1.5 m of the target zone centre.

Center-start navigation achieves 100% success across all four zones. From inside a room, 80% success is achieved; the failure case in 1 of 5 trials was caused by accumulated odometry drift after 10+ minutes of prior manual movement testing. Mid-corridor navigation achieves 60% success, reflecting the additional geometric complexity of the room-exit routing phase when the robot is misaligned relative to the target corridor gap.

### C. Voice Command Recognition

Table IV presents command recognition accuracy across three test conditions ($N = 50$ trials per condition, 150 total). Transcription accuracy measures Whisper output correctness; NLU accuracy measures the fraction of transcriptions that yielded a correct command JSON; end-to-end (E2E) accuracy measures successful robot action execution.

TABLE IV
VOICE COMMAND RECOGNITION ACCURACY ($N = 50$ PER CONDITION)

| Condition | Transcr. | NLU | E2E |
|---|---|---|---|
| Quiet (no noise) | 96% | 98% | 94% |
| Background noise (55 dB) | 82% | 91% | 76% |
| Accented speech | 78% | 94% | 73% |

The NLU accuracy exceeding transcription accuracy (e.g., 94% vs. 78% for accented speech) directly quantifies the Layer 2 recovery contribution of Groq LLaMA 3.3 70B: inputs garbled by the ASR are correctly interpreted by the LLM via semantic inference. The gap between NLU and E2E accuracy (94% vs. 73% for accented speech) reflects navigation execution failures unrelated to voice recognition.

Notably, NLU accuracy under noise (91%) exceeds transcription accuracy (82%) by 9 percentage points, confirming Layer 2 recovers a significant fraction of noise-corrupted transcriptions. The most common noise-induced failure was truncated commands (e.g., "go to" without the destination zone) caused by background noise masking final syllables. These were correctly flagged as *unknown* by Layer 1 and resolved by LLM context inference in 9 of 12 such cases.

For comparison, Layer 1 alone (without LLM fallback) achieves only 42% NLU accuracy on accented speech — a 52 percentage point gap versus the dual-layer system. This quantifies the indispensable contribution of Layer 2 for accent-diverse deployments.

### D. API Efficiency

Table V compares API call frequency between VRC-7 and a hypothetical full-cloud baseline (Groq LLM called on every transcription).

TABLE V
API CALL EFFICIENCY COMPARISON

| Metric | Full-Cloud | VRC-7 |
|---|---|---|
| LLM calls / 10 commands | 10 | $\approx 1.5$ |
| LLM calls / minute | $\approx 20$ | $\approx 3$ |
| Time to quota exhaustion | $\approx 4$ min | $\approx 26$ min |
| Offline operation | Degraded | Full local NLU |

### E. System Assessment

Table VI provides a structured multi-criteria assessment using a 0–6 scoring scale, where 0 = criterion completely unmet and 6 = criterion fully met. Each criterion is defined against a stated requirement.

TABLE VI
SYSTEM ASSESSMENT (SCALE 0–6; 0=UNMET, 6=FULLY MET)

| Criterion | Score | Evidence |
|---|---|---|
| Nav. accuracy (center) | 6 | 100%, all 4 zones |
| Nav. accuracy (arbitrary) | 4 | 60–80%; drift-limited |
| Voice acc. (quiet) | 5 | 94% E2E |
| Accent robustness | 4 | 73% E2E, 94% NLU |
| System responsiveness | 4 | 0.3–0.5 s latency |
| Wall collision avoidance | 5 | LiDAR + routing |
| Continuous listening | 5 | 0 dropped commands |
| API efficiency | 5 | 85% call reduction |
| Graceful offline degradation | 5 | Local NLU + faster-whisper |
| **Weighted mean** | **4.8/6** | |

## VI. CHALLENGES AND SOLUTIONS

### A. WSL2 Audio Subsystem Configuration

**Challenge:** The ALSA audio backend in WSL2 timed out on `sounddevice.InputStream` initialisation with error `PaErrorCode -9987`. This was caused by ALSA attempting to open the audio device before the PulseAudio socket was established.

**Solution:** Setting `os.environ['PULSE_SERVER'] = 'unix:/mnt/wslg/PulseServer'` at Python module import time (before `sounddevice` initialises its PortAudio backend) routes all audio through the WSLg PulseAudio server. This solution is specific to WSLg 1.0.65+ and must be applied before the first `import sounddevice` statement.

**Lesson:** Environment variables affecting C-level libraries (PortAudio) must be set at the OS level before the library is loaded into the process, not after `import`.

### B. Groq API Rate Limiting

**Challenge:** The Groq free tier rate limit (≈30 req/min combined across Whisper and LLaMA) was exhausted within 4 minutes of initial testing, where every audio chunk triggered both a transcription and an NLU call.

**Solution:** Three complementary optimisations were implemented: (1) the dual-layer NLU architecture reduces LLM calls to ≈15% of inputs; (2) the two-stage VAD cascade reduces transcription calls by ≈60%; (3) a 60-second automatic Groq restoration timer reactivates cloud NLU after a quota hit.

**Generalisation:** This challenge and solution pattern is broadly applicable to any ROS2 system integrating cloud AI APIs under rate-limited free tiers.

### C. Corridor Wall Collisions from Arbitrary Positions

**Challenge:** Time-based dead-reckoning navigation from a fixed start position produced straight-line paths that intersected corridor walls at non-gap positions when the robot started mid-corridor or inside a room. The robot would collide, its odometry would become invalid, and subsequent navigation would fail entirely.

**Solution:** The 4-step room-exit routing algorithm was developed. The key insight is that all valid navigation paths in this world can be decomposed into at most four axis-aligned segments that exclusively pass through the two corridor gaps. By computing waypoints that always route through gap positions ($x = \pm 6$), the algorithm guarantees wall-free traversal from any starting position.

### D. Accent Mis-transcription as Foreign Language

**Challenge:** Groq Whisper, without language constraints, interpreted accented English as European languages. "Turn right" was transcribed as "Törn rät" (Icelandic/Danish), "stop" as "Stopp" (German/Norwegian), and "pharmacy" as "farmasi" (Malay). The local regex NLU correctly returned *unknown* for these inputs, but without Layer 2, commands were silently dropped.

**Solution:** Three measures were combined: (1) `language='en'` forces English-only decoding; (2) the vocabulary prompt biases the ASR prior; (3) Groq LLaMA 3.3 70B interprets garbled outputs by intent. The combination elevates accented NLU accuracy from 42% (no mitigations) to 94% (all three mitigations applied).

### E. TurtleBot3 Physics Instability at High Speed

**Challenge:** At $v = 0.9$ m/s (the initially targeted operating speed), the TurtleBot3 Burger model tipped over in Gazebo during acceleration and sharp turns. This corrupted the `/odom` transform, causing all subsequent odometry-based navigation to fail with increasingly large positional errors.

**Solution:** Operating speed was reduced to $v_{max} = 0.5$ m/s and turn rate to $\omega_{max} = 1.5$ rad/s. Post-reduction, no tipping events were observed across 200+ test navigation runs. This constraint is documented as a boundary condition of the system (Section IV).

## VII. Discussion

### A. Why LLaMA 3.3 70B for NLU?

The choice of LLaMA 3.3 70B over smaller models was empirically motivated. During development, LLaMA 3.1 8B was first evaluated on the same garbled-input test set (50 accent-distorted transcriptions). LLaMA 3.1 8B achieved 67% correct NLU output versus 94% for LLaMA 3.3 70B — a 27 percentage point gap. Analysis of LLaMA 3.1 8B failures revealed two recurring patterns: (1) the smaller model selected plausible but incorrect interpretations for phonetically ambiguous inputs (e.g., "farmasi" → "farm" rather than pharmacy); (2) it occasionally hallucinated command structures not defined in the system prompt. LLaMA 3.3 70B's larger parameter count provides superior in-context instruction following, critical for reliably adhering to the JSON schema even on highly distorted inputs. The tradeoff is higher per-call latency (LLaMA 3.3 70B: ≈200–400 ms; LLaMA 3.1 8B: ≈80–120 ms via Groq), which is acceptable given that Layer 2 is invoked for only ≈15% of commands.

### B. Comparison with Keyword-Based Approaches

The local regex NLU Layer 1 is functionally equivalent to keyword-based systems used in prior hospital robotics work [1], [5]. On the clean-audio test set, Layer 1 alone achieves 91% accuracy, comparable to or exceeding published keyword-matching systems. The addition of Layer 2 raises accented-speech accuracy from 42% (Layer 1 alone) to 94% (dual-layer), a 52 percentage point improvement — the central quantitative contribution of this work.

### C. Limitations

**Odometry drift:** The primary navigation limitation is dead-reckoning without SLAM. Over extended sessions (>15 min), odometry drift reduces mid-corridor navigation success from 60% to approximately 30%. This is a fundamental property of wheel-encoder odometry; the solution is SLAM-based localisation (ROS2 Nav2 with AMCL), documented as a priority future extension.

**API dependency:** Accent handling beyond the compiled regex fallback patterns requires Groq API availability. Production deployment would require either a paid Groq tier (no rate limits) or local LLM inference. LLaMA 3.3 70B requires ≈40 GB GPU VRAM at float16 precision — feasible on modern workstations but incompatible with the TurtleBot3's Raspberry Pi 3B onboard compute. Quantised variants (GGUF 4-bit, ≈20 GB) via llama.cpp or Ollama are a viable near-term solution on a companion GPU workstation.

**Single-speaker evaluation:** The voice recognition results were obtained from two speakers with Indian English accents. Generalisation to other accent groups (e.g., East Asian English, African English, European English) has not been empirically validated and represents a limitation of the current evaluation scope.

**Simulation gap:** All navigation and voice results were obtained in simulation. Physical robot deployment introduces additional challenges: wheel slip on real flooring, variable

LiDAR reflectivity, microphone placement constraints, and latency variability in real network conditions.

### D. Future Applications and Extensions

VRC-7 establishes a foundation for several high-impact extensions:

1) **Real-world deployment:** The voice pipeline is hardware-agnostic. Replacing the Gazebo simulation with a physical TurtleBot3 requires only updating the `/odom` and `/scan` topic sources. SLAM-based localisation (ROS2 Nav2 with AMCL) should replace odometry-only navigation for production use.
2) **Multilingual support:** The Groq LLaMA NLU prompt can be trivially extended to support commands in multiple languages (German, French, Hindi) — directly applicable to the multilingual clinical staff composition in European hospitals.
3) **Multi-robot coordination:** Multiple VRC-7 instances can share a hospital map via ROS2 namespacing. The voice interface can be extended with robot-addressing commands ("Robot A, go to ICU") for fleet management scenarios.
4) **Integration with hospital information systems:** The navigation zones can be linked to FHIR-compliant patient location data, enabling destination commands such as "deliver to Room 412" mapped through an EHR lookup.
5) **Multimodal fusion:** Combining voice commands with visual context (e.g., gesture recognition via the onboard camera) would further improve command disambiguation in noisy environments [13].

## VIII. Conclusion

This paper presented VRC-7, a hybrid local-cloud AI pipeline for accent-robust voice control of an autonomous hospital delivery robot in ROS2. The system's two primary technical contributions — the dual-layer NLU architecture and the 4-step room-exit routing algorithm — directly address the dominant failure modes of prior voice-controlled hospital robot systems: accent-induced NLU failures and navigation wall collisions from arbitrary starting positions.

Experimental results demonstrate:

- 100% navigation success from the center position across all four hospital zones.
- 94% NLU accuracy for accented speech, a 52 percentage point improvement over Layer 1 alone.
- 85% reduction in LLM API calls compared to full-cloud NLU, enabling sustained operation on free API tiers.
- Graceful degradation to full local operation (local regex NLU + `faster-whisper`) when cloud APIs are unavailable.

The system demonstrates that the dichotomy between accent robustness and API efficiency is not fundamental — a carefully designed hybrid pipeline achieves both simultaneously. We believe this architecture is broadly applicable beyond hospital robotics to any voice-controlled robotic system that must operate under API rate constraints while serving diverse-accent user populations.

The complete source code, Gazebo world model, and evaluation scripts are openly available at https://github.com/Mprabhu26/turtlebot_vrc.


## Acknowledgment

The authors thank **Prof. Dr. Peter Nauth** for his expert supervision, continuous guidance, and invaluable feedback throughout the conception and development of this work. This project was completed as part of the Autonomous Intelligent Systems course, Semester 3, Winter 2025/26, at Frankfurt University of Applied Sciences, Germany.


## References


[1] J. Pineau, M. Montemerlo, M. Pollack, N. Roy, and S. Thrun, "Towards robotic assistants in nursing homes: Challenges and results," *Robotics and Autonomous Systems*, vol. 42, no. 3–4, pp. 271–281, 2003.

[2] World Health Organisation, "Hearing loss due to recreational exposure to loud sounds: a review," WHO Press, Geneva, 2015. [Online]. Available: https://www.who.int/docs/default-source/documents/noise/noise-guidelines.pdf

[3] M. Ahn, A. Brohan, N. Brown, et al., "Do as I can, not as I say: Grounding language in robotic affordances," in *Proc. Conference on Robot Learning (CoRL)*, 2022.

[4] M. Perera, V. Ranasinghe, and L. Liu, "Framework for integrating large language models with a robotic health attendant for adaptive task execution in patient care," *Applied Sciences*, vol. 14, no. 21, p. 9922, 2024.

[5] J. E. Domínguez-Vidal and A. Sanfeliu, "Voice command recognition for explicit intent elicitation in collaborative object transportation tasks: a ROS-based implementation," in *Companion Proc. ACM/IEEE International Conference on Human-Robot Interaction*, pp. 412–416, 2024.

[6] K. Chatzilygeroudis et al., "A voice-enabled ROS2 framework for human-robot collaborative inspection," *Applied Sciences*, vol. 14, p. 4138, 2024.

[7] OperateLLM Authors, "OperateLLM: Integrating Robot Operating System (ROS) tools in large language models," in *Proc. IEEE Conference*, 2024.

[8] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, eabm6074, 2022.

[9] Z. Xie et al., "Large language model-based task planning for service robots: A review," *arXiv preprint arXiv:2510.23357*, 2024.

[10] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in *Proc. 40th Int. Conf. Machine Learning (ICML)*, vol. 202, pp. 28492–28518, PMLR, 2023.

[11] Silero Team, "Silero VAD: pre-trained enterprise-grade voice activity detector," GitHub, 2021. [Online]. Available: https://github.com/snakers4/silero-vad

[12] SYSTRAN, "faster-whisper: Faster Whisper transcription with CTranslate2," GitHub, 2023. [Online]. Available: https://github.com/SYSTRAN/faster-whisper

[13] H. Chen, M. C. Leu, and Z. Yin, "Real-time multi-modal human–robot collaboration using gestures and speech," *Journal of Manufacturing Science and Engineering*, vol. 144, no. 10, p. 101007, 2022.